CaneSense AI

Demo Guide

Five-minute video script and live-demo checklist

STUDENT	**Raj Tripathi**
ROLL NUMBER	**16014223062**
BATCH	**B1 AIDS**
PROFESSOR	**Sagar Korde**
USE CASE	**KJS-AGR-01**
PROJECT	**CaneSense AI: AI-Powered Precision Irrigation Intelligence for Sugarcane Farms**

Optional companion to the consolidated report. All data shown in the demo is synthetic.

CaneSense AI — Five-Minute Video Presentation Script

Target length: 5:00. Start the app before recording (`python -m streamlit run app.py`), open <http://localhost:8501> in a 1440×900 browser window, and confirm the scenario is **Normal Conditions** (use *Reset to Normal* in the Scenario Simulator if not).

0:00 – 0:25 · Introduction (to camera or over the Overview screen)

> "Hello Professor Sagar Korde. I am **Raj Tripathi**, roll number **16014223062**, batch **B1 AIDS**, a final-year student. This is my Cloud Computing mini-project for the assignment *Cloud Computing Experiments through Somaiya Vidyavihar AI Use Cases*. My project is **CaneSense AI: AI-Powered Precision Irrigation Intelligence for Sugarcane Farms**, use case **KJS-AGR-01**. All data you will see is synthetic."

0:25 – 1:15 · Executive Overview

Screen: Overview.

> "CaneSense AI helps a farm manager decide where and when to irrigate. The banner shows the demo-mode and system-health indicators — the health check is computed, not decorative, and the cloud chip says *not configured* because I have not connected a cloud service."
>
> "Five metrics are calculated from the dataset: 6 villages with 28 plots, **13 plots needing irrigation**, **10 high-risk plots**, **16,807 cubic metres** of estimated water demand, and 27 of 28 sensors online."
>
> "Here is the field map with synthetic coordinates. Triangles are high urgency, diamonds medium, circles low, so risk is never colour alone. On the right is the priority queue, then the moisture trend, urgency by village, and weather."

Action: click one red triangle on the map; point at the highlighted selection.

1:15 – 1:55 · Farm Intelligence and the AI advisor

Screen: Farm Intelligence → type `Jaggery` *in search → click a row.*

> "The table filters by plot, village, urgency, crop stage and sensor status. Selecting a plot shows live readings, a 21-day moisture history against its irrigation trigger, the model's probability, and a recommendation with water volume, duration and plain-language reasons."

Screen: AI Irrigation Advisor.

> "The recommendation has two parts: a scikit-learn random-forest predicts urgency — macro-F1 of about 0.9 on a held-out synthetic test split — and a separate transparent rule engine chooses the action and calculates water depth, volume and duration. I am clear that these scores only measure how well the model recovers my synthetic rule, not real-farm accuracy."

1:55 – 3:05 · Scenario Simulator (the main demo)

Screen: Scenario Simulator.

> "Now the before-and-after demonstration. I select **Simulate Dry Spell** and click *Preview impact*: high-urgency plots would go from 10 to 21 and water demand from about 16,800 to 35,400 cubic metres."

*Action: click **Apply Scenario**; wait for the progress steps.*

> "Applying runs the model again on every plot. The summary shows the change, and an entry is added to the scenario history with a timestamp."

Screen: Overview.

> "Back on the Overview, the KPIs, map colours, queue and charts have all updated — high-risk plots are now 21 and demand is 35,436 cubic metres. Ten plots are deferred because the shared water budget is exhausted."

*Screen: Simulator → Preview **Rain Expected** → Apply.*

"With **Rain Expected**, the advice changes to *Rainfall adjustment recommended* and demand falls to about 12,900 cubic metres. The presets are deterministic, so this repeats exactly. *Reset to Normal* restores the baseline."

3:05 – 3:40 · Operations and Reports

Screen: Irrigation Operations.

"Operations turns recommendations into a priority queue and a suggested pump schedule. I can mark a plot as completed, which simulates irrigation and updates the model. No real equipment is controlled."

Screen: Reports & Exports → choose a village → Generate PDF report → download PDF, CSV, Excel.

"Reports use exactly the selected records. The PDF has a branded header, risk overview, charts, a table with repeated headers, page numbers and a synthetic-data disclaimer; CSV and Excel match the same scope."

3:40 – 4:40 · Cloud experiments

Screen: Cloud Lab.

"The Cloud Lab shows my four experiments honestly. **Experiments 7 and 8 were executed.** For right-sizing I benchmarked the real analytics workload at 1, 2, 4 and 8 CPUs: 11 seconds on one CPU, 6.9 on two, 5.0 on four and 4.2 on eight. Speed-up is only 2.6 times for eight times the CPUs, because only model training parallelises. Peak memory was about 485 megabytes whatever the CPU count."

"For FinOps I used the same measurements with assumed list prices. These are **estimates, not actual charges**. Two vCPUs is the best balance; the largest instance is fastest but costs about twenty times more per run than the smallest suitable one."

"**Experiments 2 and 3 — Docker and Kubernetes** — are fully prepared: a Dockerfile with health check, and Kubernetes manifests with probes, resource limits, a service and an autoscaler. On this machine Docker and kubectl are not installed, so I have *not* run them, and the Cloud Lab says so. The exact commands are in the experiment guides."

4:40 – 5:00 · Close

Screen: Methodology.

"To summarise: a deployable agricultural intelligence application with a real ML model, a transparent recommendation engine, deterministic scenarios, exportable reports, automated tests, and documented cloud experiments. The limitations are clear: synthetic data, no field validation, and Docker and Kubernetes still to be executed. Thank you, Professor Korde."

Timing tips

- Pre-load every page once so charts render instantly.
- If a click is missed, press *Reset to Normal* and repeat — scenarios are deterministic.
- Keep the Dry Spell numbers handy: High 10 → 21, demand 16,807 → 35,436 m³, 10 deferred.

CaneSense AI — Live Demo Checklist

Presenter: Raj Tripathi · Roll 16014223062 · Batch B1 AIDS · Prof. Sagar Korde · Use case KJS-AGR-01

Before you start (5 minutes)

- ■ `python -m pytest -q` shows all tests passing
- ■ `python -m streamlit run app.py` running; <http://localhost:8501> loads without a red error box
- ■ Browser window 1440×900, zoom 100%, bookmarks bar hidden, notifications off
- ■ Sidebar visible; header chips read *Demo mode · synthetic data*, *System healthy*, *Scenario: Normal Conditions*
- ■ Hero photograph visible (no broken image); map shows 28 markers
- ■ `outputs/` folder open in a second window for the PDF/Excel downloads
- ■ Internet off is fine — nothing in the core demo needs it (street-tile map toggle excluded)

Demo flow

1. [ ] **Overview** — read the five KPIs (13 need irrigation, 10 high-risk, 16,807 m³, 27/28 sensors)
2. [ ] Click a plot on the map; mention shape + colour for risk
3. [ ] **Farm Intelligence** — search `Jaggery`, select a row, show history chart, prediction, recommendation, sensor health
4. [ ] **AI Irrigation Advisor** — open a card, explain model vs rule engine, note the limitations line
5. [ ] **Scenario Simulator** — Preview **Simulate Dry Spell** (10 → 21 high, 35,436 m³) → **Apply Scenario**
6. [ ] **Overview** again — KPIs, map colours and queue changed
7. [ ] Simulator → **Rain Expected** → Apply (rainfall-adjustment advice, 12,928 m³)
8. [ ] Simulator → **Sensor Failure** → Apply (7 offline; Farm Intelligence shows *Sensor readings need review*)
9. [ ] **Irrigation Operations** — set a plot to *Completed*, **Apply status changes**, show moisture/KPI effect
10. [ ] **Reports & Exports** — choose a village, **Generate PDF report**, download PDF, CSV, Excel; open the PDF
11. [ ] **Cloud Lab** — show the four status cards (2 and 3 *not executed*, 7 and 8 *executed*), right-sizing and FinOps charts
12. [ ] **Methodology** — metrics, confusion matrix, limitations
13. [ ] Simulator → **Reset to Normal**

If something goes wrong

Symptom	Fix
Page shows an error box	Refresh the browser tab; state resets to Normal
Scenario seems stuck	Simulator → *Reset to Normal*
Map blank in Street-tiles mode	Switch to *Field-zone (offline)*
Port 8501 busy	`python -m streamlit run app.py --server.port 8502`
Model missing	`python scripts\train_model.py`

Honesty reminders

- Say "synthetic" when showing any number. Do not call the model accurate on real farms.
- Say "estimated" for all costs. Say Docker and Kubernetes were **not executed** on this machine.